

Automated Incident Root-Cause Analysis via Close-Code Classification

Week 5 – Project Analysis and Outcome Explanation

1. Introduction

The objective of this project is to develop an intelligent machine learning-based system capable of predicting the root cause of IT incidents using historical incident management data. The project focuses on automating incident root-cause analysis through close-code classification.

The dataset used in this project is the Incident Management Process Enriched Event Log dataset derived from a real-world ServiceNow-based IT incident management environment.

The primary target variable in this project is:

- `closed_code`

This represents the final resolution or root-cause category assigned to an incident after it has been resolved.

2. Problem Statement

In large-scale IT systems, thousands of incidents are generated daily. Manually identifying the root cause of incidents is time-consuming, resource-intensive, and highly dependent on expert knowledge.

The goal of this project is to:

- Automatically predict the probable root cause category of incidents
- Reduce manual diagnostic effort
- Improve incident response efficiency
- Support IT service management teams using intelligent recommendations

This problem is challenging because:

- Similar incidents may have different root causes
 - Root causes often overlap
 - Incident descriptions are short and ambiguous
 - The dataset lacks detailed technical logs and diagnostics
-

3. Dataset Description

Dataset Used:

- Incident Management Process Enriched Event Log

Original Dataset Shape:

- 141,712 rows
- 36 columns

The dataset contains:

- Incident states
- Priorities
- Assignment groups
- Symptoms
- SLA details
- Timestamps
- Resolution information
- Close codes

The dataset is event-log based, meaning multiple records exist for a single incident during different stages of its lifecycle.

4. Major Challenges Faced

4.1 Event Log Complexity

The dataset contains multiple rows for the same incident because every update creates a new event.

Solution:

- Aggregated the dataset to incident-level records by keeping the final closed state.
-

4.2 Imbalanced Dataset

Some close-code classes contained thousands of records, while others contained very few.

Example:

- Close code 6 had more than 14,000 records
- Some classes had fewer than 10 records

Problem:

- Machine learning models become biased toward majority classes.

Solution:

- Removed extremely rare classes
 - Applied balanced sampling
 - Used SMOTE oversampling techniques
-

4.3 Weak Feature Relationships

The dataset does not contain:

- System logs
- Error traces
- Diagnostic reports
- Technical RCA notes

As a result, the mapping between symptoms and close codes is highly ambiguous.

Example:

"Email not working" could be caused by:

- Network failure
- Authentication issue
- Server issue
- Configuration issue

This creates significant class overlap.

5. Data Preprocessing

Several preprocessing techniques were applied to improve data quality.

5.1 Incident Aggregation

Only final closed incidents were retained.

This reduced noise and transformed the dataset from event-level to incident-level.

Result:

- Dataset reduced from 141,712 rows to 24,918 incident records.
-

5.2 Cleaning Target Variable

The `closed_code` column contained values such as:

- code 6
- code 7

These were converted into numeric labels.

Invalid labels such as `?` were removed.

5.3 Rare Class Removal

Very low-frequency classes were removed because they:

- introduced instability
- reduced learning quality
- created severe imbalance

Only classes with sufficient samples were retained.

5.4 Balanced Sampling

Balanced sampling was performed so that all classes contained equal records.

Final balanced dataset:

- 130 records per class

- 10 classes
- Total: 1300 records

This ensured fair learning.

6. Feature Engineering

Feature engineering was one of the most important stages of the project.

6.1 Time-Based Features

The `opened_at` timestamp was decomposed into:

- opened hour
- opened day
- opened month
- weekday
- weekend indicator
- business-hours indicator

These features helped identify temporal incident patterns.

6.2 Feature Interaction Engineering

Combined features were created such as:

- impact + urgency
- priority + category
- incident state + priority

These interaction features improved pattern learning.

6.3 Frequency Encoding

High-cardinality categorical columns were transformed using frequency encoding.

Examples:

- assignment_group
- cmdb_ci
- location

This reduced dimensional explosion caused by one-hot encoding.

6.4 TF-IDF Text Processing

The `u_symptom` textual column was processed using TF-IDF vectorization.

Techniques used:

- unigrams
- bigrams
- trigrams
- stop-word removal

This converted textual symptoms into machine-readable numerical vectors.

Result:

- 145 TF-IDF features generated.
-

7. Feature Selection

Wrapper-based feature selection was applied using Logistic Regression with L1 regularization.

Purpose:

- remove noisy features
- reduce dimensionality
- improve generalization

Result:

- Final features reduced to 300 optimized features.
-

8. Machine Learning Models Used

The following machine learning algorithms were trained and evaluated.

8.1 Logistic Regression

Advantages:

- Simple
- Fast
- Good baseline model

Result:

- Accuracy: 55.77%
 - F1 Score: 55.00%
-

8.2 Linear SVM

Advantages:

- Excellent for high-dimensional sparse data
- Performs well with TF-IDF features

Result:

- Accuracy: 57.69%
 - F1 Score: 56.78%
-

8.3 Decision Tree

Advantages:

- Interpretable
- Rule-based learning

Result:

- Accuracy: 45.38%
- F1 Score: 45.29%

Observation:

- Decision Trees overfit easily on sparse high-dimensional data.

8.4 Random Forest

Advantages:

- Ensemble learning
- Better generalization
- Handles nonlinear relationships

Result:

- Accuracy: 58.46%
- F1 Score: 57.82%

This became the best-performing model.

8.5 XGBoost

Advantages:

- Gradient boosting
- Strong optimization capability

Result:

- Accuracy: 56.15%
 - F1 Score: 55.21%
-

9. Why Random Forest Performed Best

Random Forest performed best because:

- It combines multiple decision trees
 - Reduces overfitting
 - Handles feature interactions well
 - Works effectively on mixed structured and sparse features
 - Captures nonlinear relationships between incident patterns and close codes
-

10. Why Accuracy Did Not Reach 90%

The accuracy did not reach 90% because the problem itself is inherently ambiguous.

Main reasons:

10.1 Lack of Diagnostic Features

The dataset does not contain:

- logs
- traces
- stack information
- server metrics

These are critical for exact root-cause identification.

10.2 Class Overlap

Multiple close codes share similar symptoms.

Example:

"Email not working"
may correspond to:

- network issue
- server issue
- authentication issue
- configuration issue

This makes exact prediction difficult.

10.3 Human Labeling Variability

Different engineers may assign different close codes for similar incidents.

This introduces label inconsistency.

10.4 Multi-Class Complexity

The model predicts among 10 different classes.

Multi-class classification is significantly harder than binary classification.

11. Industry-Level Improvements Applied

The project incorporated several advanced industrial techniques.

Techniques Used:

- Balanced sampling
- SMOTE oversampling
- TF-IDF NLP processing
- Feature interaction engineering
- Wrapper-based feature selection
- Ensemble learning
- Frequency encoding
- Model comparison
- Top-K prediction system

These techniques significantly improved model performance.

12. Top-K Root Cause Prediction

A Top-K prediction system was implemented.

Instead of predicting only one root cause, the system predicts:

- Top 3 most probable close codes

This is closer to real-world ITSM systems.

Benefits:

- Improves practical usability
 - Assists support engineers
 - Reduces diagnostic uncertainty
-

13. Final Outcome

Final Best Model:

- Random Forest

Final Performance:

- Accuracy: 58.46%
- F1 Score: 57.82%

These results are considered strong for real-world incident root-cause prediction problems.

14. Conclusion

This project successfully developed an intelligent machine learning system for automated incident root-cause classification using real-world ITSM incident data.

Advanced preprocessing, NLP techniques, feature engineering, balancing, and ensemble learning significantly improved prediction performance.

Although exact close-code prediction remains challenging due to overlapping symptoms and limited diagnostic features, the system achieved strong practical performance and demonstrates the feasibility of AI-assisted incident management.

The project reflects realistic industrial conditions and showcases the application of machine learning in IT service management automation.

15. Future Scope

Possible future improvements include:

- Deep learning sequence models (LSTM/Transformer)
 - BERT embeddings for better symptom understanding
 - Hierarchical root-cause prediction
 - Integration with real-time ServiceNow systems
 - Inclusion of system logs and monitoring data
 - Explainable AI for root-cause reasoning
-

16. Final Statement

This project demonstrates how machine learning and NLP techniques can support automated incident root-cause analysis in enterprise IT environments by assisting engineers with intelligent close-code recommendations.